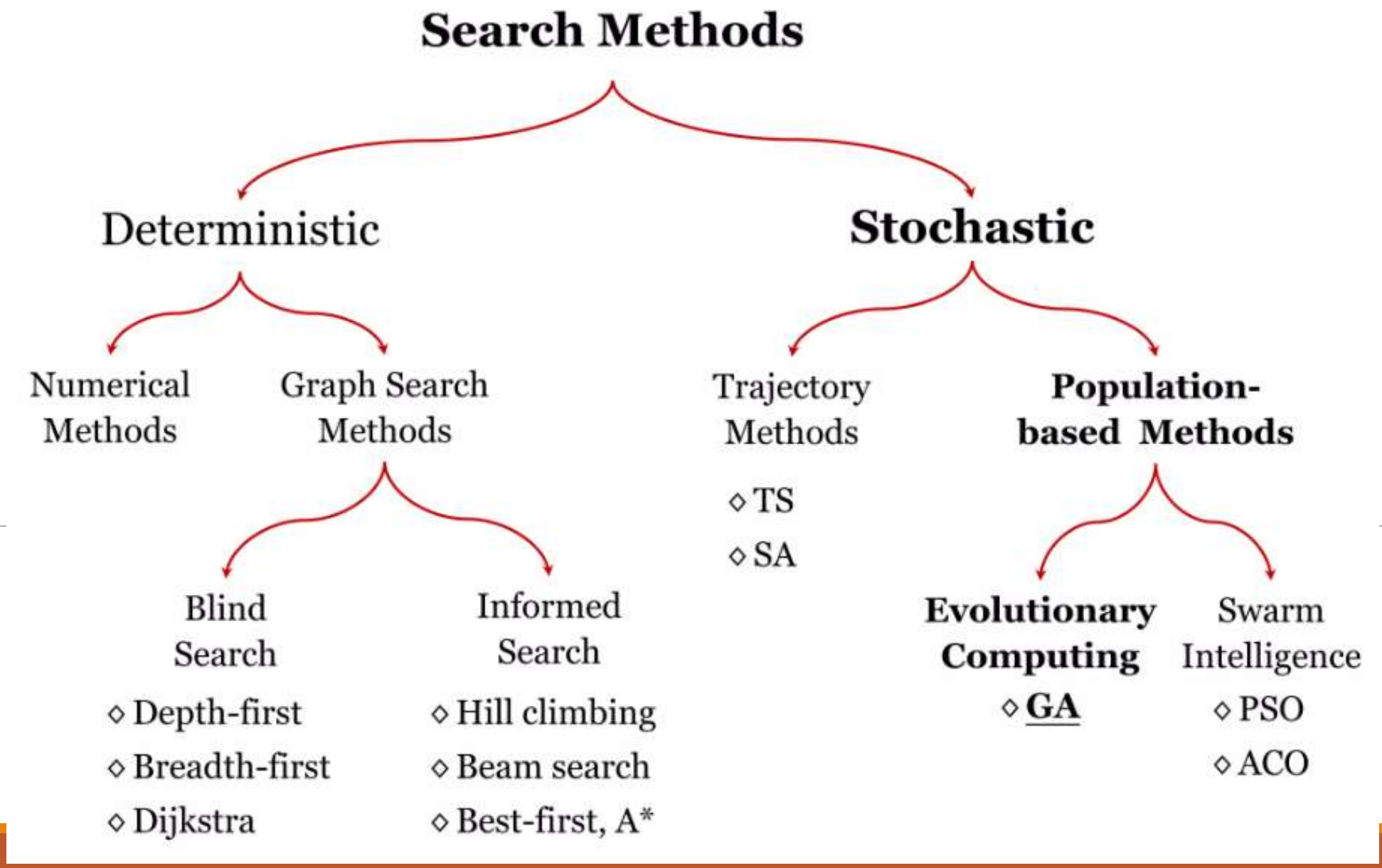
Genetic Algorithm



Genetic Algorithm

- Developed by John Holland, University of Michigan (1970's):
  - ◇ To understand the adaptive processes of natural systems.
  - ◇ To design artificial systems software that retains the robustness of natural systems.

John Holland
(1929-)
American scientist

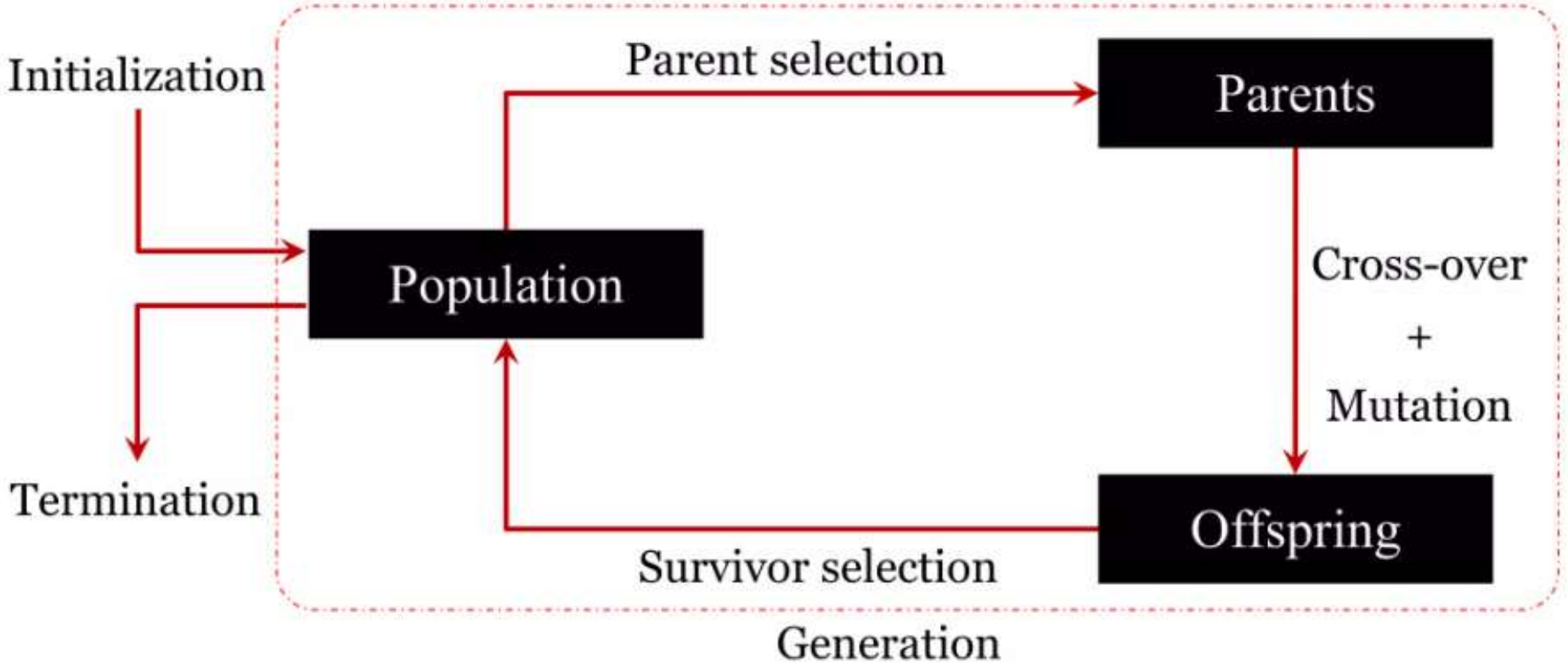
- The original form of the GA, as illustrated by John Holland in 1975 had distinct features: **a bit string representation**, **proportional selection** and **cross-over** to produce new individuals.
- Several variations to the original GA have been developed using different representation schemes, selection and reproduction operators.

Genetic Algorithm

- **Basic Idea:**
  - ◇ A class of probabilistic optimization algorithms
  - ◇ Inspired by the biological evolution process
  - ◇ Uses concepts of "Natural Selection" and "Genetic Inheritance" (Darwin 1859).
  - ◇ Take a population of candidate solutions to a given problem.
  - ◇ Use operators inspired by the mechanisms of natural genetic variation.
  - ◇ Apply selective pressure toward certain properties.
  - ◇ Evolve a more fit solution.

Genetic Algorithm

- **The Algorithm:**



Steps in the Genetic Algorithm:

1. **Initialization →**
    1. The algorithm **starts** with a **random population** of possible solutions (individuals).
2. **Parent Selection →**
    1. The **best individuals** (parents) are selected based on their fitness (how good they are at solving the problem).
3. **Crossover & Mutation →**
    1. **Crossover:** The selected parents **combine their traits** (genes) to create new solutions (offspring).
    2. **Mutation:** Some small **random changes** are introduced to increase diversity and avoid getting stuck in bad solutions.
4. **Offspring Generation →**
    1. The newly created **offspring** replace weaker individuals in the population.
5. **Survivor Selection →**
    1. The algorithm chooses the **best individuals** from both parents and offspring to form the next generation.
6. **Iteration (Generation Loop) →**
    1. Steps **2-5 repeat** for multiple generations until the best solution is found.
7. **Termination →**
    1. The process **stops** when the algorithm meets a **stopping condition**, such as:
        1. The best solution **doesn't improve** after many generations.
        2. A **fixed number of generations** is reached.
        3. The solution reaches a **desired quality level**.

Genetic Algorithm

- Solution to a problem solved by genetic algorithms, is evolved
- Algorithm is started with a set of solutions (represented by **chromosomes**) called **Population**
- Solutions from one population are taken and used to form a new population for a better one.
- Solutions which are selected to form new solutions (**offspring**) are selected according to their **fitness** - the more suitable they are the more chances they have to **reproduce**
- Repeated until some condition is satisfied.

Genetic Algorithm

- **The Algorithm:**

Initialize population with random candidates,

Evaluate all individuals,

While *termination criteria is not met*

    Select parents,

    Apply crossover,

    Mutate offspring,

    Replace current generation,

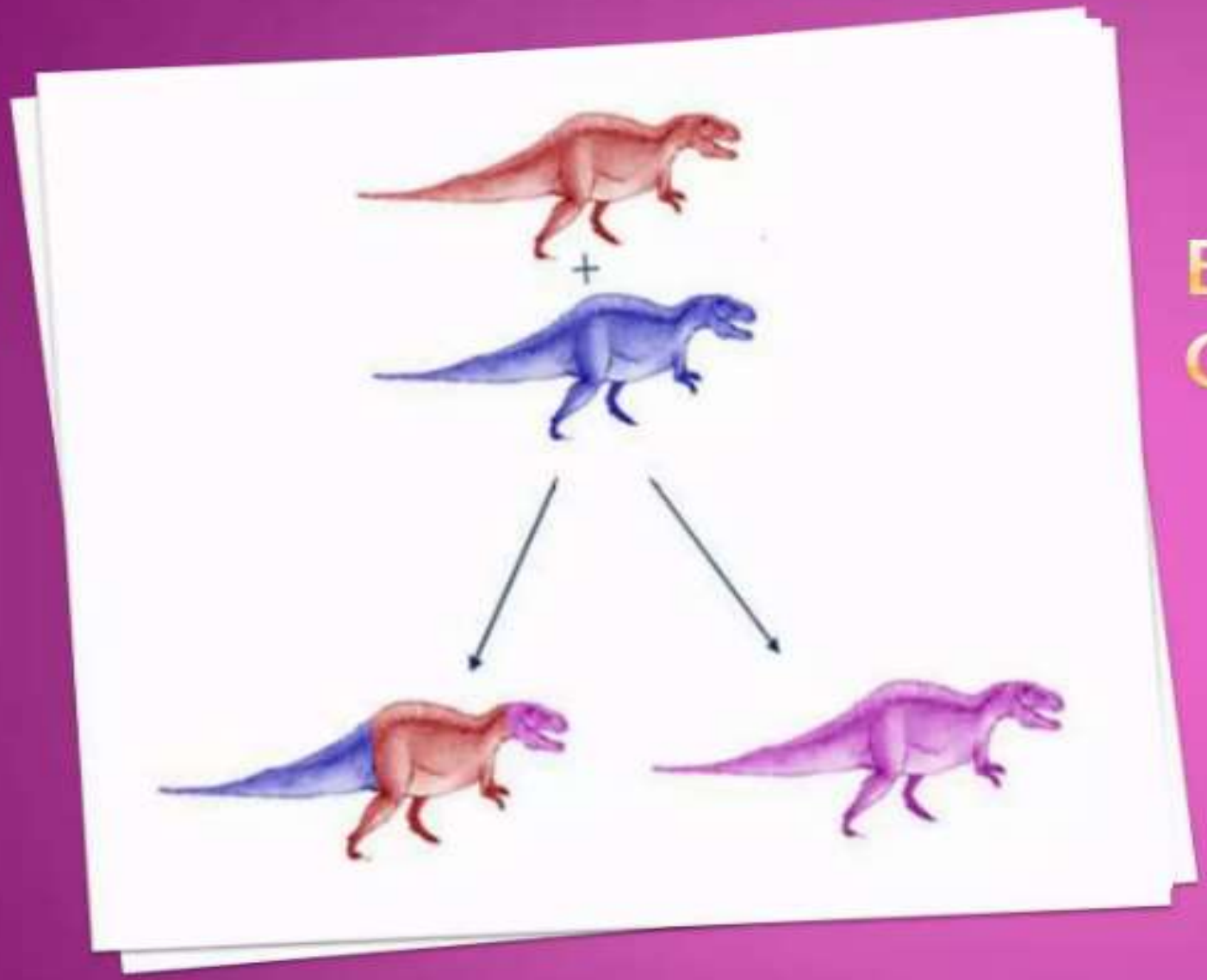
end while

Genetic Algorithm

- **The Algorithm:**

  The **termination criteria** could be:

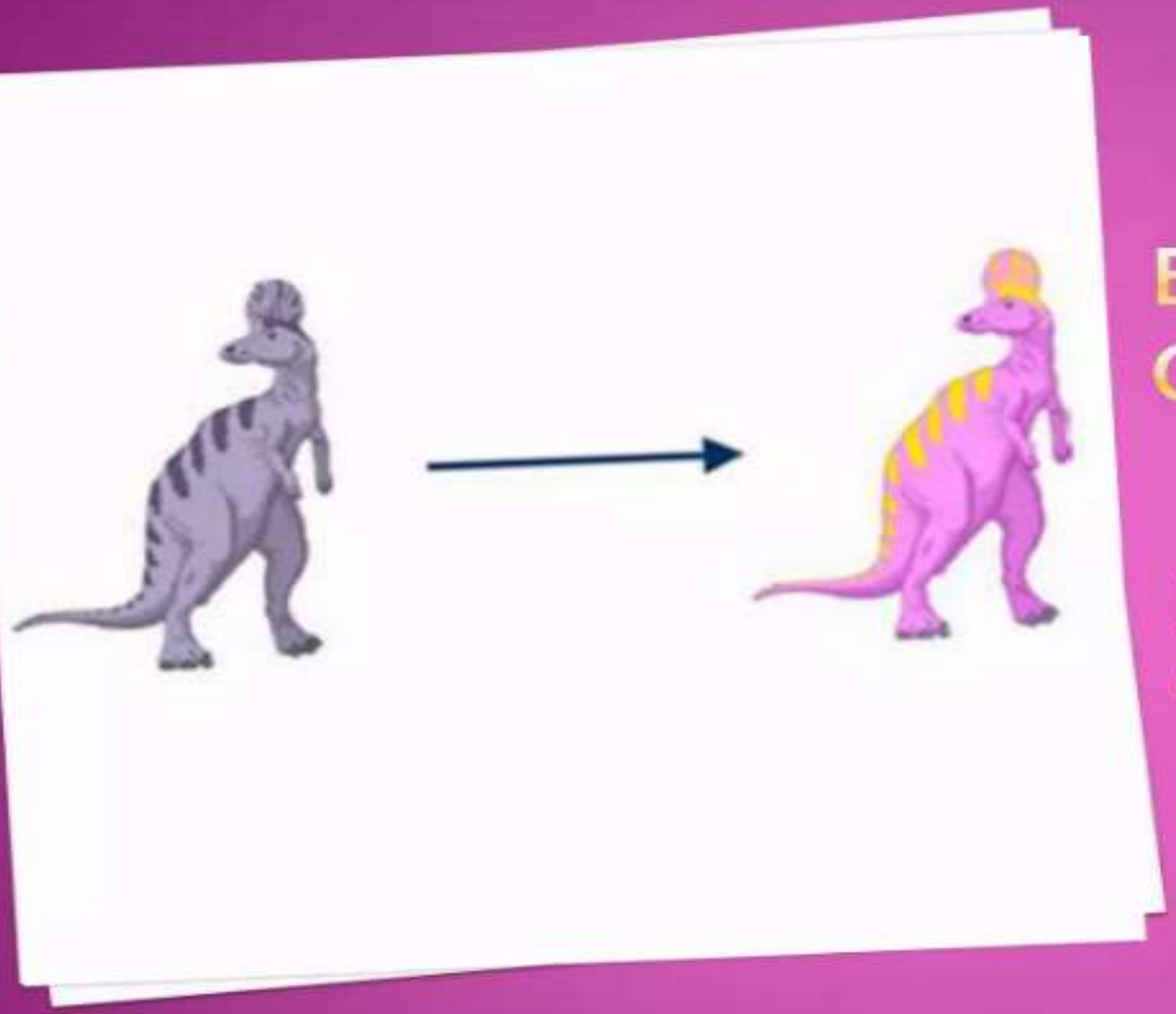
  - A specified number of generations or fitness evaluations (100 or 150 generations),
  - A minimum threshold reached,
  - No improvement in the best individual for a specified number of generations,



EVOLUTION DUE TO GENETIC CROSSOVER

Offsprings have combinations of features inherited from each parent

Image adapted from http://www.wpdipart.com



EVOLUTION DUE TO GENETIC MUTATION

Random changes are observed

Image adapted from http://www.wpdipart.com

Encoding in Genetic Algorithms (GA)

Encoding in Genetic Algorithms (GA) refers to the way solutions (individuals) are represented in a format that the algorithm can process. Since GAs work with populations of solutions, each solution needs to be encoded as a **chromosome (or genotype)** before applying selection, crossover, and mutation.

Encoding

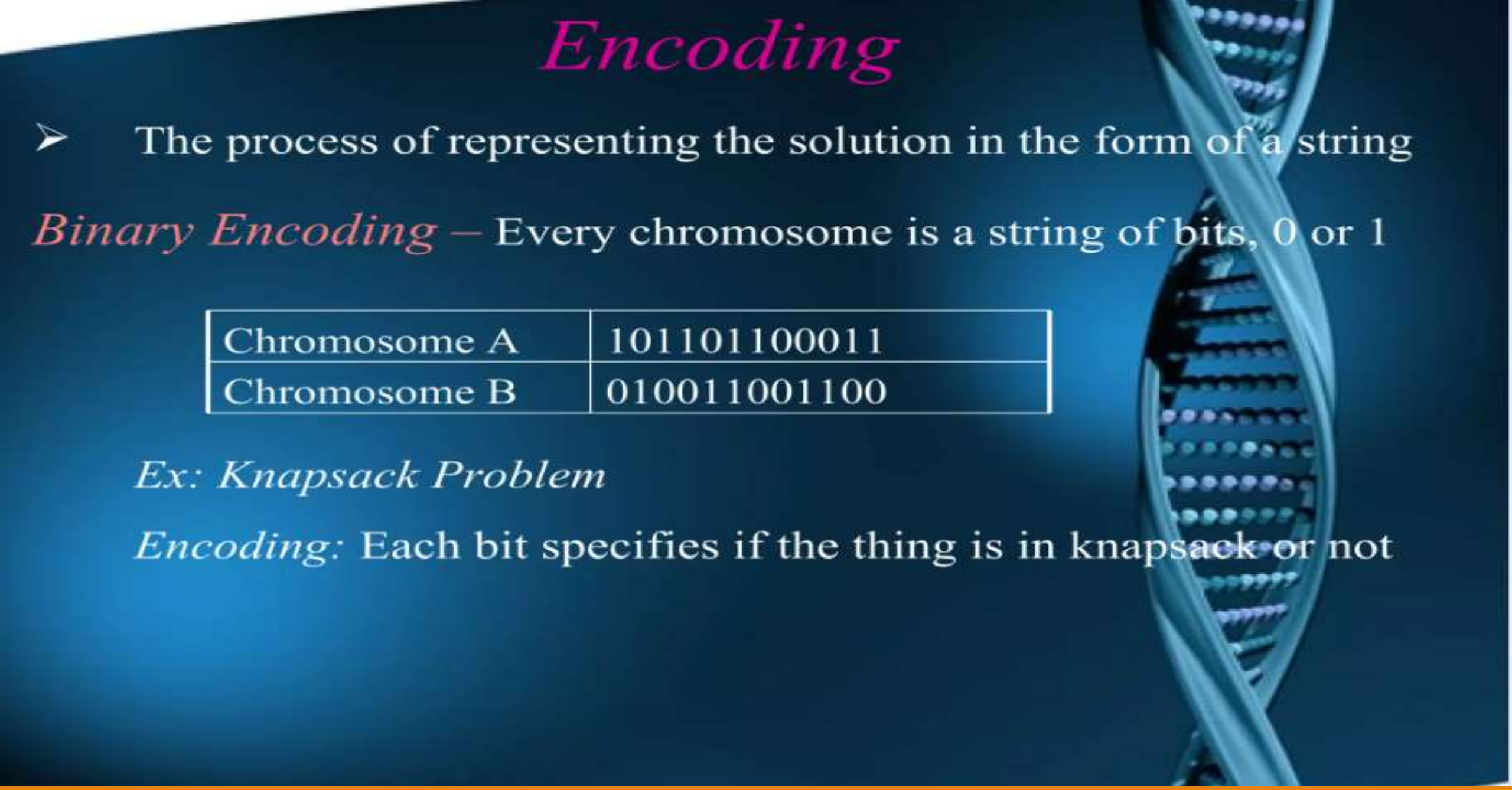
- The process of representing the solution in the form of a string

Binary Encoding – Every chromosome is a string of bits, 0 or 1

Chromosome A	101101100011
Chromosome B	010011001100

Ex: Knapsack Problem

Encoding: Each bit specifies if the thing is in knapsack or not



Permutation Encoding – Every chromosome is a string of numbers, which represents the number in the *sequence*. Used in ordering problems.

Ex: Traveling Sales Person Problem

Encoding: Chromosome represents the order of cities, in which the salesman will visit them

Chromosome A	1 5 3 2 6 4 7 9 8
Chromosome B	8 5 6 7 2 3 1 4 9

Value Encoding in Genetic Algorithm (GA)

Value encoding is a way to represent solutions in a **numeric form** instead of binary (0s and 1s). It is useful when the solution involves **real numbers, decimal values, or symbols**.

What is Value Encoding?

- Each chromosome is a **string of real numbers or symbols**.
- Used when binary encoding is not suitable (e.g., in complex optimization problems).

Value encoding

Neural Network Weight Optimization

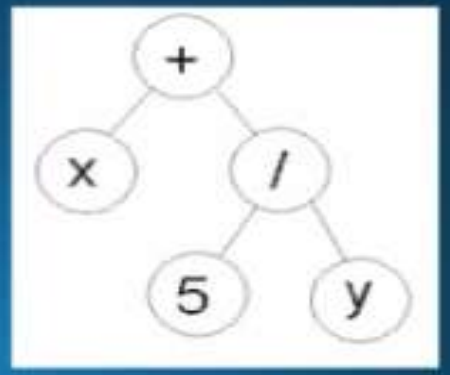
•Chromosome stores real-valued weights:

csharp

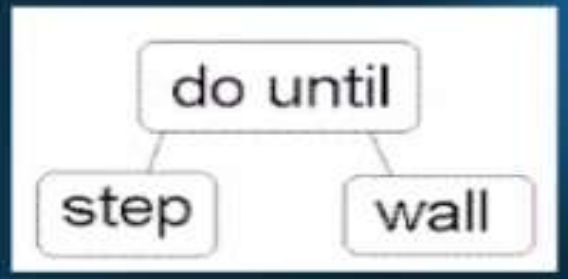
[0.25, -0.85, 0.63, 1.23, -0.44]

Tree Encoding – Every chromosome is a tree of some objects, such as functions or commands in a programming language

- Used for evolving programs or expressions in Programming Languages



(+ x (/ 5 y))



(do_until step wall)

- *Ex:* Finding a function from given values

The problem: Some input and output values are given. Task is to find a function, which will give the best output to all inputs

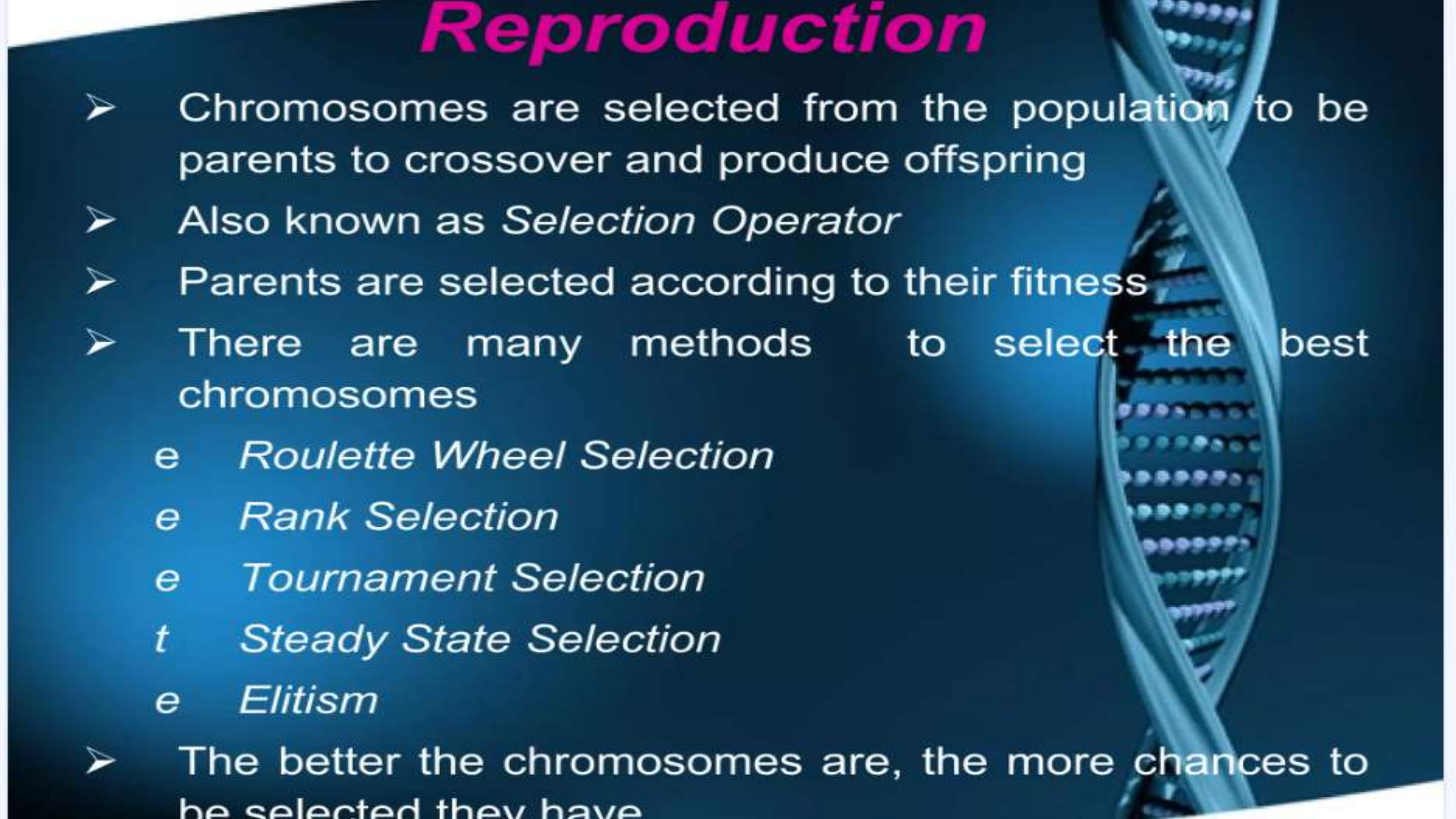
GA operators

Genetic Algorithms (GAs) use **three main operators** to evolve better solutions:

1. **Selection (Choosing Parents)**
2. **Crossover (Combining Solutions)**
3. **Mutation (Introducing Small Changes)**

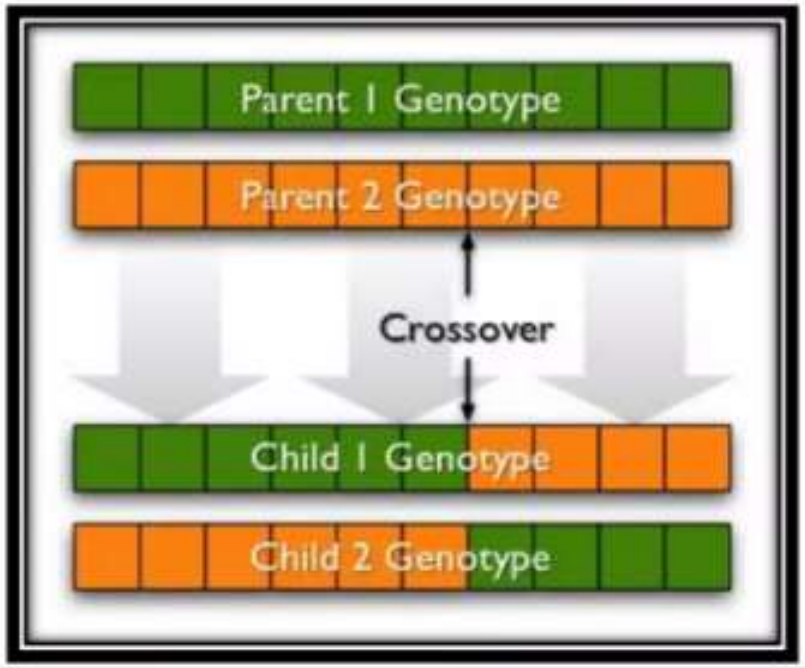
Reproduction

- Chromosomes are selected from the population to be parents to crossover and produce offspring
- Also known as *Selection Operator*
- Parents are selected according to their fitness
- There are many methods to select the best chromosomes
  - e *Roulette Wheel Selection*
  - e *Rank Selection*
  - e *Tournament Selection*
  - t *Steady State Selection*
  - e *Elitism*
- The better the chromosomes are, the more chances to be selected they have

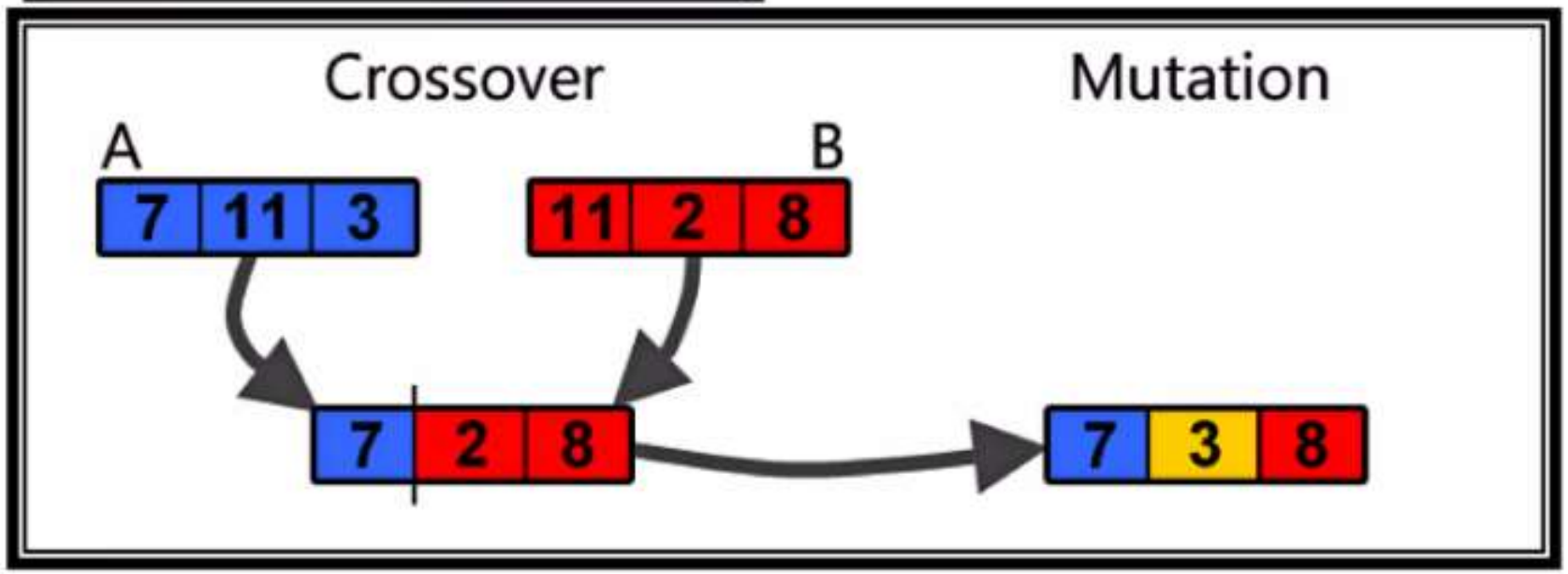


Crossover

- Applied to create a better string
- Proceeds in three steps
  1. The reproduction operator selects at random a pair of two individual strings for mating
  2. A cross-site is selected at random along the string length
  3. The position values are swapped between two strings following the cross-site
- *MAY NOT ALWAYS* yield a better or good solution
- Since parents are good, probability of the child being good is high
- If offspring is not good, it will be removed in the next iteration during “Selection”



Here crossover and mutation are shown diagrammatically. As evident from the diagrams crossover is the selection of bits for the purpose of **"exploitation"** whereas mutation serves the purpose of **"exploration".**



Single Point Crossover:

- A cross-site is selected randomly along the length of the mated strings
- Bits next to the cross-site are exchanged
- If good strings are not created by crossover, they will not survive beyond next generation

Contd..

Two-point Point Crossover:

- Two random sites are chosen
- The contents bracketed by these are exchanged between two mated parents

Contd..

Uniform Crossover:

- Bits are randomly copied from the first or from the second parent
- A random mask is generated
- The mask determines which bits are copied from one parent and which from the other parent

- **Mask: 0110011000 (Randomly generated)**

Parent 1	1 0 1 0 0 0 1 1 1 0
Parent 2	0 0 1 1 0 1 0 0 1 0
Offspring 1	0 0 1 1 0 0 1 0 1 0
Offspring 2	1 0 1 0 0 1 0 1 1 0

Mutation

- Restores lost genetic materials
- Mutation of a bit involves flipping it, changing 0 to 1 and vice versa